

AMZ Driverless

The Full Autonomous Racing System

BY: SARA WAEL MOUSA



Perception

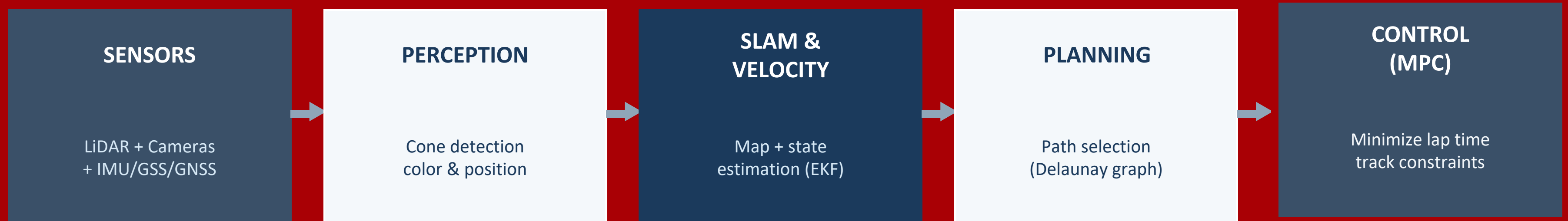
Velocity Est.

SLAM

Planning

Control

System Architecture Overview



SLAM Mode (Lap 1)

Track is unknown. Simultaneously build map & navigate using local cone observations only. Speed limited to 3 m/s.

Localization Mode (Laps 2–10)

Track is known. Localize in the map and push to performance limits (MPC). No further map updates.

⚠ Sensor Redundancy is a Core Design Principle

Two independent perception pipelines (LiDAR + Camera) and multiple velocity sensors ensure the system can cope with any single sensor failure — validated repeatedly in competition.

Key Challenge: Track is completely unknown at race start. All computation is on-board.

Perception — LiDAR Pipeline

1. Field of View Trimming

Filter rear-facing points
(car body occludes rear)

2. Motion Distortion Correction

LiDAR spins while car moves →
fuses velocity to undistort scan
→ prevents "ghost cones"

3. Ground Removal

Adaptive algo segments FoV
into angular/radial bins.
Fits regression line → removes ground pts

4. Euclidean Clustering + Reconstruction

Cluster non-ground pts. Reconstruct
cylinder around each cluster center
(ground removal removes cone pts too!)

5. Legality Filter

Check expected point count
 $E(d) = f(\text{distance, cone size, LiDAR resolution})$

6. Color Estimation (CNN)

Yellow: high-low-high intensity
Blue: low-high-low intensity
4-conv + 5-FC layers → 96% accuracy

Critical: Motion Distortion — LiDAR sweeps 360° while the car moves. Without correcting for vehicle velocity, points from different sweep angles are captured at different car positions, warping the point cloud and creating false "ghost cones."

Perception — Camera Pipeline

Shared Step

YOLOv2 Object Detection
Detects cone bounding boxes, classifies Blue / Yellow / Orange

MONOCULAR PIPELINE

Keypoint Regression (CNN)

ResNet backbone predicts 7 keypoints on each detected cone in the image patch (80×80 px input)

PnP Algorithm

Matches 2D image keypoints to known 3D cone geometry. Solves for camera→cone transform using RANSAC-PnP to handle noisy correspondences

Output: 3D Cone Position

Pose estimate for each cone relative to car frame. Higher variance than stereo.

STEREO PIPELINE

Bounding Box Propagation

Keypoints from left frame projected to 3D via PnP, then re-projected onto right camera plane using epipolar constraints

SIFT Feature Matching

Matches features between left & right bounding boxes. SIFT chosen over BRISK: fewer mismatches at acceptable compute cost

Triangulation → 3D Position

Triangulate matched feature pairs. Median filtering for robustness. Lower variance than monocular. RMSE $\approx$ 0.25 m

Result: Camera error < 1.0 m at 15 m range | Stereo variance lower than mono | Map RMSE = 0.25 m (vision-only)

Velocity Estimation — Extended Kalman Filter

WHY?

Undistort LiDAR scans
(motion compensation)

Propagate pose in SLAM

Feed control module
(MPC needs state)

Single sensors are noisy,
biased, or fail entirely

Wheel spin $\neq$ velocity
(slip up to 30% at
high acceleration)

6 SENSORS FUSED

GSS (Ground Speed Sensor)

GNSS / GPS

IMU (accel + gyro)

Wheel Speed (4×)

Motor Torques

Steering Angle

EKF: The Fusion Engine

Maintains a 9-state estimate: velocities (v_x , v_y), yaw rate (r), accelerations, and 4 wheel slip ratios.

Predict step: propagate state using vehicle + tire model (Pacejka).
Update step: correct with each sensor measurement asynchronously.

Chosen for computational efficiency with mildly nonlinear systems.

Failure Detection

Outliers (spikes): Chi-square test on each new measurement. Rejects instantaneous spikes.

Drift (gradual failure): Variance-based detector across multiple sensors measuring same state. Discards sensor with highest variance contribution.

Both run together — chi alone misses slow drift; variance alone needs multiple sensors.

Result: Position drift < 0.5% over 310 m track | Accurate even with 30% wheel slip

SLAM & Mapping — FastSLAM 2.0

The Problem: Track is unknown. Cones have no unique IDs. The system must simultaneously locate itself AND build a cone map — neither is possible without the other. → SLAM (Simultaneous Localization and Mapping).

Data Association	FastSLAM 2.0 (Particle Filter)	Lap Closure & Localization
Each new cone observation must be matched to an existing landmark in the map	Maintains M particles, each carrying an entire hypothesis of: car pose + full cone map	Closure detected when: particle spread < threshold, heading ≈ start, position ≈ start
Nearest-neighbour matching using Mahalanobis distance (accounts for uncertainty)	Particles are weighted by how well new observations match the map	After closure: best-weight particle's map is frozen as ground truth
If no close match → new landmark initialized	Inherently handles multi-hypothesis data association	Mode switches SLAM → Localization (Monte Carlo)
Uncertain associations are the core challenge	Scales linearly in landmarks (vs quadratic EKF-SLAM)	Position RMSE vs ground truth: 0.2 m over 230 m track

Two-stage sensor failure detection in SLAM: (1) per-landmark confidence counters to flag dubious landmarks; (2) sensor-level rejection if <80% of observations match high-confidence landmarks.

Planning & Control

PLANNING (SLAM Mode)

1 Delaunay Triangulation

Observed cones → vertices of triangulation. Mid-points of cone pairs form a discrete path graph. Naturally encodes "drive between cones."

2 Beam Search Tree Growth

Grow a tree of candidate paths through the graph (breadth-first, worst paths pruned). Re-run every 50 ms with fresh cone observations.

3 Cost Function Scoring

Score each path on: max angle change, track width std deviation, cone spacing regularity, color rule compliance (blue left / yellow right), and path length vs sensor range.

Best path → pure pursuit controller (Lap 1 only)

CONTROL (MPC, Localization Mode)

Why not just follow the center line?

A racing driver cuts corners to minimize arc length. MPC finds the optimal trajectory that respects the track boundary — it naturally drives a racing line.

Vehicle Model (Blended Bicycle)

Dynamic bicycle model (nonlinear tire forces via Pacejka) for $v_x > 5$ m/s. Kinematic model for slow speeds (avoids singularity at $v \approx 0$). Linearly blended in between.

MPC Formulation

Maximize progress along reference path (minimize lap time). Minimize contouring error. Subject to: track boundary constraints (circular approximation), tire friction ellipse constraints, input rate limits.

Testing Framework & Competition Results

WHY SIMULATE FIRST?

Real-world testing is expensive & slow

Time, manpower, safety margins. At 100 km/h the car leaves the track before a human reacts.

ATS: Automated Testing System (FSSIM)

Triggered on every pull request + overnight runs. First-principles physics model. Cone noise simulated.

Logging & Analysis

PostgreSQL + RViz 3D visualizations, web interface. Binary pass/fail + lap stats per simulation.

Test extreme cases

Push to handling limits: aggressive driving, hard layouts, poor weather — go to the limit, not beyond.

RACE RESULTS — FSG 2018

L1 28.913s	L2 28.608s	L3 28.558s	L4 28.483s	L5 28.625s
L6 28.542s	L7 28.523s	L8 28.571s	L9 28.512s	L10 28.593s

28.48s

Fastest Lap

±0.13s

Lap Consistency

1.5g

Peak Lateral Accel

285.93s

Total Time (1st)

Key Lessons Learned

- ① Real-world testing is irreplaceable — simulation finds bugs early, but hardware reveals everything
- ② Fixed, well-defined module interfaces allow independent development and testing of each subsystem
- ③ Delays (~300 ms end-to-end) are the biggest bottleneck to matching human driver performance

Conclusion

01

Redundancy is Non-Negotiable

Dual perception pipelines + multi-sensor fusion. Every single sensor failed at some point in testing. The system survived because no single failure was critical.

02

Two-Phase Racing Strategy

SLAM mode (slow, careful, mapping) → Localization mode (fast, optimized). The map built in Lap 1 is the foundation for all subsequent performance.

03

End-to-End Integration Matters

Each module (perception, EKF, SLAM, planning, MPC) has clean interfaces but the full pipeline must be tested together — integration reveals what unit tests miss.

04

Simulation Multiplied Testing Capacity

ATS enabled finding good parameters in simulation, fine-tuning at the track. Automated overnight runs caught regressions before competition.

1st Place · FSG 2017 · FSI 2018 · FSG 2018

Open-source: github.com/AMZ-Driverless

Video: youtube.com/watch?v=hYpToBIMpVQ